

159.735 Assignment 5

Lattice points in a hypersphere

You have been commissioned to help design a new quantum warp drive engine. Since this engine is designed to drive spaceships through hyperspace, you will need to deal with spatial dimensions greater than 3.

Write a GPU program to count lattice points within a N -dimensional hypersphere.

Background

Given an N -dimensional sphere of radius, r , centred at the origin of an N -dimensional coordinate system, first complete a sequential function to count the number of integer coordinate points inside the sphere. Here are some examples to allow you to check your work:

1. A 1-dimensional sphere of radius 25.5 has 51 integer coordinate points:
 $\{(\pm 25, \pm 24, \pm 23, \dots, \pm 1, 0)\}$
2. A 2-dimensional sphere of radius 2.05 has 13 integer coordinate points within the sphere:
 $\{(0, 0), (-1, 0), (-2, 0), (1, 0), (2, 0), (-1, -1), (-1, 1), (1, -1), (1, 1), (0, -2), (0, -1), (0, 1), (0, 2)\}$
3. A 3-dimensional sphere of radius 1.5 has 19 integer coordinate points inside the sphere:
 $\{(-1, 0, 0), (-1, 0, 1), (-1, 0, -1), (-1, -1, 0), (-1, 1, 0), (1, 0, 0), (1, 0, 1), (1, 0, -1), (1, -1, 0), (1, 1, 0), (0, 0, 0), (0, 1, 0), (0, 1, 1), (0, 1, -1), (0, -1, 0), (0, -1, 1), (0, -1, -1), (0, 0, -1), (0, 0, 1)\}$

The assignment

A small start up program will be provided, where you are asked to complete the sequential function `count_inside`.

Having completed the sequential program, then develop an implementation to run on your GPU.

The number of dimensions and the hyper-sphere radius are to be user input parameters, i.e. something like:

```

int main(int argc, char* argv[]) {
    const int ndim = std::atoi(argv[1]);
    const double r = std::atof(argv[2]);

    const ULONGINT num = h_count_in(r, ndim);

    ...
}

```

You will need to decide what your kernel function will do and how to organize your threads. It is also likely you will encounter some limitations on your graphics card that will affect performance and the values for `r` and `ndim` that will actually work.

Submission

Please submit your C or C++ source code together with a brief report. In your report you should address at least the following. Marks will be awarded for discussions that demonstrate clear understanding on GPU architectures.

- State the graphics card you used together with any relevant specs.
- Briefly describe your algorithm for counting the number of integer coordinates within an N -sphere
- Provide a description on how you parallelized the above problem.
- Present appropriate performance data, and comment on your result. In particular, assess the limitations of your device when dealing with this computation problem.

This assignment is worth 20% of your final grade

Appendix: Coding Hints

To convert a decimal integer number, `num`, into a number in base `B`, you can use a number of possible algorithms, for example:

```

idx = 0;
while (num != 0) {
    rem = num % B;
    num = num / B;
    digit[idx] = rem;
    ++idx;
}

```

Examples: $1208_{10} = 10111000_2$, $1208_{10} = 1122202_3$, $1208_{10} = 2270_8, \dots$.

You can use `cudaMalloc()` to create a single data word, not just arrays, on the GPU device. Also, you can use `cudaMemset()` to initialize a device value and you can use `cudaMemcpy()` to copy single data words from the device to the host.

You will probably encounter a situation where a number of threads can modify a single value. Clearly, this would have to be done atomically. The CUDA library provides a (overloaded) function `atomicAdd()` that makes this easy.

Further background

This assignment is the Gauss Circle Problem generalized to any number of dimensions. This is a well studied problem in number theory

https://en.wikipedia.org/wiki/Gauss_circle_problem

If you are mathematically inclined, see for example

[https://awwalker.com/2017/03/07/
lattice-points-in-high-dimensional-spheres](https://awwalker.com/2017/03/07/lattice-points-in-high-dimensional-spheres)